PRODUCT REQUIREMENTS DOCUMENT

Grid-Guard AI

Web Admin Dashboard & Mobile Field-Inspection App

AI-driven electricity-theft risk detection — product requirements for the deployment layer

Field	Value
Author	Noor Eman
Version	1.0 (Draft)
Status	For Review
Date	August 2026
Related documents	Grid-Guard AI FYP Proposal; Grid-Guard AI ML Data, Model & Validation Strategy

1. Purpose & Overview

This document specifies the product requirements for the two user-facing surfaces of Grid-Guard AI: a web admin dashboard for revenue-protection staff, and a mobile field-inspection app for linemen. The ML detection engine that powers both surfaces (risk scoring and SHAP-based explanations) is specified separately in the companion ML Data, Model & Validation Strategy document — this PRD assumes that engine exists behind a REST API and focuses entirely on what is built on top of it.

Grid-Guard AI turns a per-consumer electricity-theft risk score into two connected workflows: an office-side triage and case-assignment workflow (web), and a field verification workflow (mobile), joined by a feedback loop that improves the underlying model over time.

2. Goals

2.1 Product goals

- Give a revenue-protection officer a ranked, explainable list of at-risk consumers instead of a random or complaint-driven inspection queue.
- Give a field inspector a simple, offline-capable way to receive an assigned case, record an inspection outcome, and close the loop — without needing a desktop or constant connectivity.
- Make every flag auditable and explainable, so it can be defended to a consumer, a supervisor, or a regulator.

2.2 Non-goals for this version

- Live integration with a real DISCO billing system — this version operates on the public/synthetic benchmark data described in the ML strategy document.
- Multi-tenant support for multiple DISCOs simultaneously — single-organization scope for v1.
- Payment, billing correction, or legal case-management workflows — out of scope; this product stops at “flag, assign, verify, feed back.”

3. Target Users / Personas

Persona	Surface	Core need
Revenue Protection Officer	Web dashboard	Needs to see which consumers are highest-risk today, understand why, and assign the right ones to field staff without wasting inspection capacity on low-confidence flags.
Field Inspector / Lineman	Mobile app	Needs to know where to go, what to look for, and a fast way to record what was found — often with poor or no signal.
Circle Manager (secondary, v1.1)	Web dashboard	Needs an aggregate view across officers/feeders to judge whether the program is working, not individual case detail.

4. Scope

4.1 In scope (v1)

- Web dashboard: consumer risk queue, SHAP-based explanation view, case assignment, basic feeder map, audit log.
- Mobile app: assigned-case queue, offline-capable inspection form with photo + GPS, sync-on-reconnect.
- Shared backend API connecting both surfaces to the ML detection engine.
- Feedback loop: inspection outcomes flow back as labeled data for future model retraining (data capture only in v1; automated retraining is v2).

4.2 Out of scope (v1)

- Real-time smart-meter (AMI) streaming ingestion — v1 operates on batch/periodic data loads, consistent with the current, largely non-AMI state of Punjab's rollout described in the proposal.
- SMS/USSD channel for low-end-phone inspectors — noted as a possible v2 addition given Pakistan's device diversity, not built in v1.
- Push notifications requiring a persistent backend service — v1 uses in-app queue refresh on open/sync instead.

5. Web Admin Dashboard — Functional Requirements

Requirements are prioritized using MoSCoW (Must / Should / Could / Won't-this-version).

ID	Requirement	Priority
WEB-1	Display a ranked, filterable list of consumers by risk score, with filters for feeder, tariff class, meter type, and risk band (high/medium/low).	Must
WEB-2	For each flagged consumer, show a SHAP-based explanation panel in plain language (e.g. “consumption dropped 60% vs. seasonal baseline”), not raw SHAP values.	Must
WEB-3	Allow an officer to assign a flagged consumer as a case to a specific field inspector, with an optional note.	Must
WEB-4	Show a case list with status (Assigned / In Progress / Closed — Confirmed / Closed — False Positive / Closed — Inconclusive).	Must
WEB-5	Maintain an audit trail: every flag generated, every assignment made, and every status change, with timestamp and actor.	Must
WEB-6	Show a feeder-level map or heat-map view of where flagged consumers are concentrated.	Should
WEB-7	Show a model-performance panel: of closed cases, what % were confirmed theft vs. false positive, trended over time.	Should
WEB-8	Support basic authentication (officer login) with role separation (officer vs. circle manager view).	Should
WEB-9	Allow CSV export of the current filtered case list for offline reporting.	Could
WEB-10	Multi-DISCO / multi-tenant support.	Won't (v1)

6. Mobile App — Functional Requirements

ID	Requirement	Priority
MOB-1	Show the inspector's assigned-case queue, sorted by priority/date assigned.	Must
MOB-2	For each case, show consumer address, meter number/ID, and a one-line plain-language risk reason (from the same SHAP explanation as the web dashboard).	Must
MOB-3	Provide an inspection form: outcome selector (Confirmed theft / False positive / Inconclusive), tampering-type checklist, free-text note.	Must
MOB-4	Allow attaching a photo of the meter and auto-tagging GPS coordinates at time of inspection.	Must
MOB-5	Work fully offline: queue, form-filling, and photo capture must not require live connectivity; data queues locally and syncs when a connection is available.	Must
MOB-6	Support both English and Urdu UI text.	Should
MOB-7	Show a simple sync-status indicator (pending / synced) per case.	Should
MOB-8	Allow the inspector to view their own closed-case history.	Could
MOB-9	Push notifications for new case assignment.	Won't (v1 — requires persistent backend service, deferred)

7. Non-Functional Requirements

Category	Requirement
Explainability	Every risk flag surfaced to either a web or mobile user must be accompanied by a human-readable explanation — no bare probability score without a reason.
Offline resilience	Mobile app must retain full core functionality (view assigned cases, fill inspection form, capture photo/GPS) with zero connectivity; sync is eventual, not required at time of use.
Auditability	Every state change (flag generated, case assigned, case closed) is logged with timestamp and actor; logs are not user-editable.
Data honesty	No UI copy anywhere in either app claims real-world validated deployment accuracy; all risk-score framing is scoped to “model output based on available data,” consistent with the ML strategy document's reporting discipline.
Performance	Web dashboard risk-queue view loads in under 3 seconds for up to 10,000 consumers on a typical broadband connection.
Accessibility of language	Mobile app UI text available in Urdu, matching the field-inspector persona's likely working language.
Security (baseline)	Authenticated access only; no consumer PII (name, CNIC, phone) displayed beyond what's operationally necessary — consumer/meter ID is sufficient for the case workflow.

8. System Architecture (Reference)

Both surfaces are thin clients over a shared backend; the backend, not either front-end, owns model inference and business logic. Full architectural detail (feature engineering, model choice, explainability

pipeline) lives in the ML strategy document — summarized here only as it constrains the product surfaces.

Layer	Responsibility	Consumed by
ML inference service	Loads the trained model (XGBoost/RF), computes risk score + SHAP values per consumer	Backend API only
Backend API (REST)	Case/assignment state, audit log, auth, serves risk data to both clients	Web dashboard, mobile app
Web dashboard	Officer-facing UI: queue, explanations, assignment, map, audit view	Revenue-protection officers
Mobile app	Inspector-facing UI: case queue, offline form, photo/GPS capture, sync	Field inspectors

9. Key API Endpoints (Indicative)

Endpoint	Method	Used by	Purpose
/consumers/risk-queue	GET	Web	Ranked, filterable list of flagged consumers
/consumers/{id}/explanation	GET	Web, Mobile	Plain-language SHAP explanation for one consumer
/cases	POST	Web	Create/assign a new case to an inspector
/cases/mine	GET	Mobile	Inspector's assigned-case queue
/cases/{id}/inspection	POST	Mobile	Submit inspection outcome, photo, GPS, notes
/cases/{id}/audit-log	GET	Web	Full history of a case's state changes
/auth/login	POST	Web, Mobile	Authenticate officer or inspector

10. Screens (Indicative List)

10.1 Web dashboard

1. Login
2. Risk Queue (default landing view — WEB-1, WEB-2)
3. Consumer Detail (explanation panel, assign action — WEB-2, WEB-3)
4. Case List / Board (WEB-4)
5. Feeder Map (WEB-6)
6. Model Performance Panel (WEB-7)
7. Audit Log (WEB-5)

10.2 Mobile app

8. Login
9. My Cases (queue — MOB-1)

10. Case Detail (address, reason — MOB-2)
11. Inspection Form (outcome, checklist, photo, GPS — MOB-3, MOB-4)
12. Sync Status / History (MOB-7, MOB-8)

11. Success Metrics

- Precision@top-K of the risk queue on held-out benchmark data (primary ML metric, defined in the ML strategy document) — product-level proxy for "is triage actually better than random."
- Time from case assignment (web) to inspection outcome submission (mobile) — workflow efficiency proxy.
- % of cases with a photo + GPS-tagged outcome (data-quality proxy for the feedback loop).
- For a portfolio/demo context: a working end-to-end trace — flag generated → assigned → inspected → outcome recorded → visible in audit log — with no manual database edits required.

12. Assumptions & Constraints

- Single developer, resume/portfolio project scope — no dedicated backend/DevOps team; free-tier hosting (Vercel/Render or equivalent) is assumed, not enterprise infrastructure.
- No live DISCO data access; all consumer data in this version is the public/synthetic benchmark data described in the ML strategy document, clearly labeled as such in the UI (e.g. a visible "demo data" indicator), not presented as live production data.
- Mobile app may be scoped down to a minimal functional prototype (MOB-1 through MOB-5 only) if time is constrained, per the effort-to-payoff discussion already had for this project — the web dashboard plus ML pipeline is the primary demonstration of capability.
- Punjab's AMI/smart-meter rollout remains partial as of this writing (government-department pilot, ~100k-meter LESCO target) — the product is designed around conventional billing-data inputs, not assumed real-time smart-meter streams.

13. Risks

Risk	Mitigation
Scope creep across two full front-ends plus an ML pipeline for a solo builder	Mobile app de-scoped to Must-only requirements if timeline slips; web dashboard is the priority surface
Demo data mistaken for real deployment claims	Explicit "demo/benchmark data" labeling in both UIs; no marketing copy claims live DISCO validation
Offline-sync complexity underestimated on mobile	MOB-5 scoped narrowly to queue+form+photo local storage with simple sync-on-reconnect, not a full conflict-resolution offline framework

14. Release Plan (Indicative)

Phase	Scope
MVP	Backend API + ML inference service; Web: Risk Queue, Consumer Detail, Case List (WEB-1–WEB-4)
V1	Web: Audit Log, Feeder Map, Model Performance (WEB-5–WEB-7); Mobile: My Cases, Case Detail, Inspection Form, offline support (MOB-1–MOB-5)
V1.1 / stretch	Auth roles (WEB-8), Urdu localization (MOB-6), sync-status polish (MOB-7), CSV export (WEB-9)